

Collections

**Collection Interfaces, Collection classes, Accessing a collection
via an Iterator**

```
int x=10;
```

```
int y=20;
```

```
int z=30;
```

Arrays

```
datatype[ ] arr = new datatype[size];
```

```
Student[ ] s=new Student[10000];
```

Limitations: size is fixed, homogeneous
data

```
s[0]=new Student( );
```

```
s[1]=new Customer( );
```

Error: incompatible type

Found: Customer

Required: Student

```
Object[ ] s=new Object[10000];
```

```
s[0]=new Student( );
```

```
s[1]=new Customer( );
```

Arrays are not implemented based on data structure(no predefined methods)

Need Of Collections

An array is an indexed Collection of fixed number of homogeneous data elements.
The Main advantage of Arrays is we can represent multiple values with a single variable.
So that reusability of the code will be improved.

Limitations of Object type Arrays:

Arrays are fixed in size i.e. once we created an array with some size there is no chance of increasing or decreasing it's size based on our requirement. Hence to use arrays compulsory we should know the size in advance which may not possible always.

2) Arrays can hold only homogeneous data elements.

Ex:-

```
Student [] s = new Student [10000];
```

```
s[0] = new Student; (correct)
```

```
s[1] = new Customer(); (wrong)
```

But We can resolve this problem by using object Arrays.

```
Object [] o = new Object [10000];
```

```
o[0] = new Student();
```

```
o[1] = new Customer();
```

Arrays Concept is not implemented based on some standard data structure hence readymade method support is not available for every requirement we have to write the code explicitly. Which is complexity of programming.

Collections

Collections are growable in nature

Homogeneous and heterogeneous objects

Standard data structure

Need Of Collections

To overcome the above limitations of Arrays we should go for Collections.

Collections are growable in nature. i.e. Based on our requirement we can increase (or) Decrease the size.

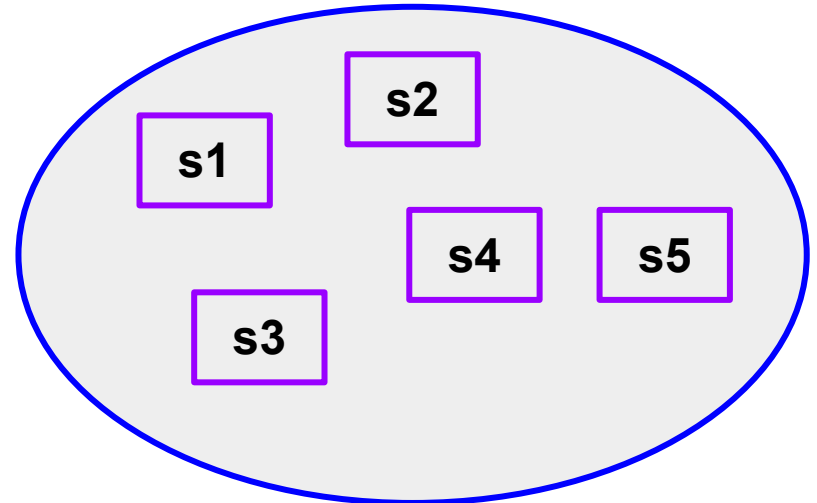
Collections can hold both homogeneous & Heterogeneous elements.

Every Collection class is implemented based on some standard data structure. Hence readymade method support is available for every requirement. Being a programmer we have to use this method and we are not responsible to provide implementation.

Collection-Group of individual objects as a single entity

To represent group of individual objects, we need several classes nad interfaces.

Collections framework - It defines **classes and interfaces** to which can be used to group objects as a single entity.



Java Collections Framework

The **Java Collections Framework (JCF)** was introduced in **J2SE 1.2** and provides a unified architecture for handling groups of objects.

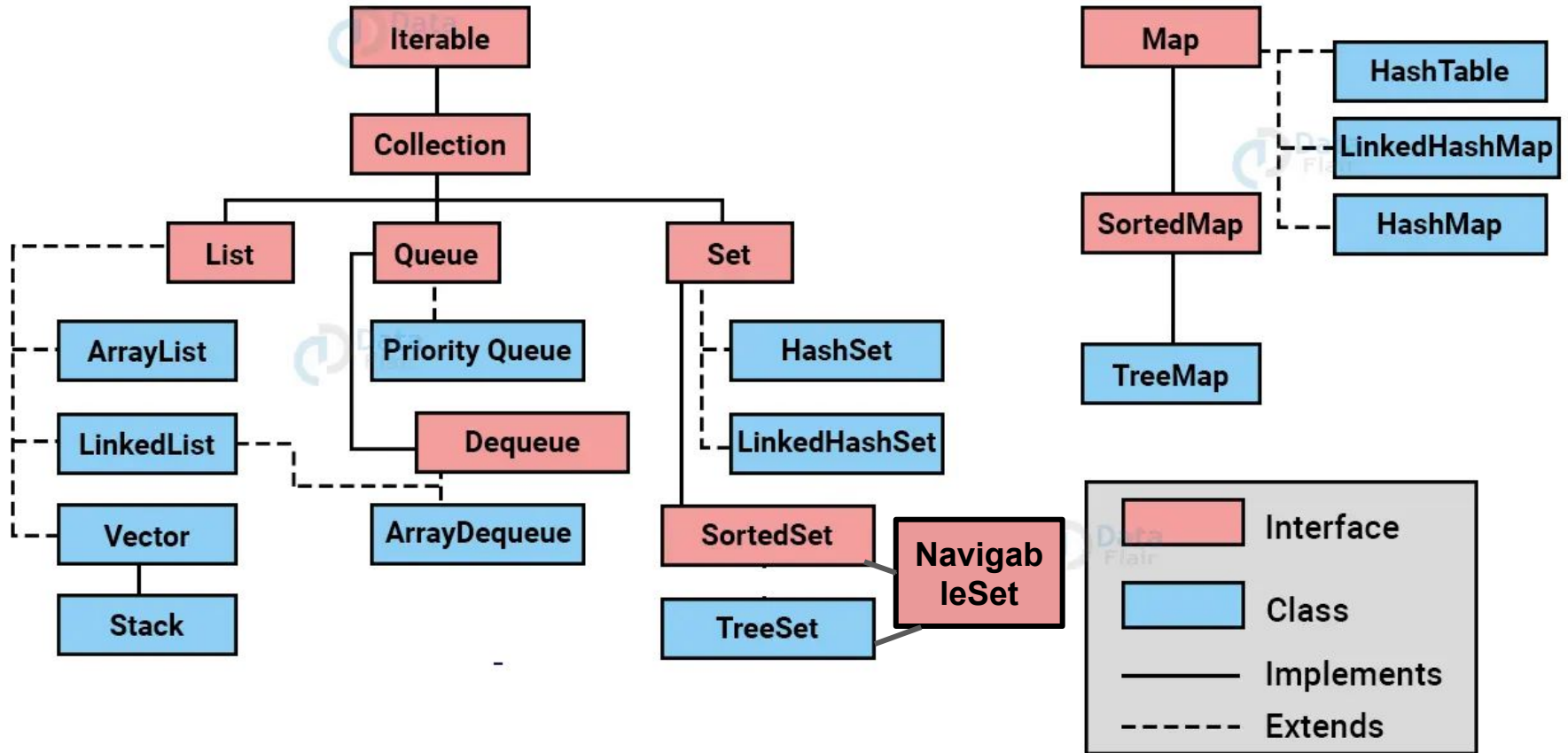
Before the Collections Framework, Java had various classes such as **Dictionary, Vector, Stack, and Properties**, each used to store collections of objects. However, these ad hoc classes **were not standardized**, and their use varied significantly, which made them difficult to extend, adapt, and maintain.

The Collections Framework addressed these issues by providing a set of standardized interfaces and implementations for managing collections of objects.

Collection Framework in Java is an architecture using which we can represent a **collection of interfaces and classes**. It is used in storing, maintaining and handling data effectively.

The **collection framework** makes manipulation of the collection easier. This makes the task of a programmer simpler.

Hierarchy of Collection Framework in Java



Goals of the Collections Framework

The **main goals** behind the design of the Collections Framework are:

1. **High performance:** The framework provides efficient implementations of fundamental data structures like dynamic arrays, linked lists, trees, and hash tables.
2. **Uniformity and interoperability:** Collections of different types (e.g., lists, sets, maps) are designed to work in a consistent manner, allowing them to interact seamlessly with one another.
3. **Extensibility:** It's easy to extend and adapt collections according to specific needs. There are predefined implementations like `LinkedList`, `HashSet`, and `TreeSet`, but you can also create your own collections if needed.
4. **Integration with arrays:** Java collections integrate with standard arrays, so you can use arrays with the framework as well.

The Collection Interfaces

The Collections Framework defines several interfaces.

Interface	Description
Collection	Enables you to work with groups of objects; it is at the top of the collections hierarchy.
Deque	Extends Queue to handle a double-ended queue. (Added by Java SE 6.)
List	Extends Collection to handle sequences (lists of objects).
NavigableSet	Extends SortedSet to handle retrieval of elements based on closest-match searches. (Added by Java SE 6.)
Queue	Extends Collection to handle special types of lists in which elements are removed only from the head.
Set	Extends Collection to handle sets, which must contain unique elements.
SortedSet	Extends Set to handle sorted sets.

Collection Interface

The Collection interface is the core of the **Java Collections Framework**. All collections in Java (like List, Set, etc.) must implement the Collection interface, which defines the basic operations for storing and manipulating groups of objects. It is a **generic interface**, meaning it can work with any object type (indicated by the type parameter E).

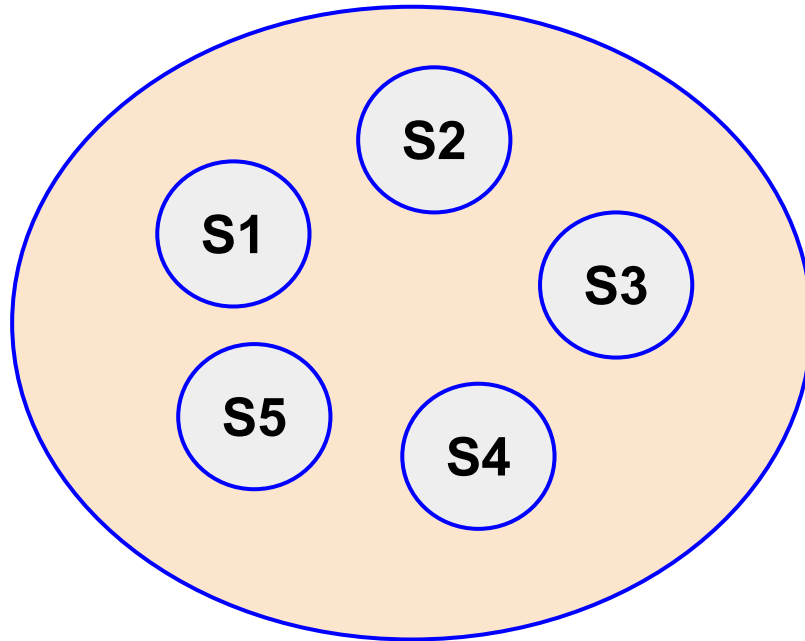
Here's the declaration:

```
interface Collection<E>
```

E represents the type of elements stored in the collection. For example, if you have **Collection<Integer>**, it will store only **Integer objects**.

Since Collection extends the **Iterable interface**, it provides a built-in mechanism for iterating through the elements, which allows collections to be used with the **enhanced for loop**.

Group of individual objects as a single entity- **Collection**



Method	Description
boolean add(E <i>obj</i>)	Adds <i>obj</i> to the invoking collection. Returns true if <i>obj</i> was added to the collection. Returns false if <i>obj</i> is already a member of the collection and the collection does not allow duplicates.
boolean addAll(Collection<? extends E> <i>c</i>)	Adds all the elements of <i>c</i> to the invoking collection. Returns true if the operation succeeded (i.e., the elements were added). Otherwise, returns false .
void clear()	Removes all elements from the invoking collection.
boolean contains(Object <i>obj</i>)	Returns true if <i>obj</i> is an element of the invoking collection. Otherwise, returns false .
boolean containsAll(Collection<?> <i>c</i>)	Returns true if the invoking collection contains all elements of <i>c</i> . Otherwise, returns false .
boolean equals(Object <i>obj</i>)	Returns true if the invoking collection and <i>obj</i> are equal. Otherwise, returns false .
int hashCode()	Returns the hash code for the invoking collection.

<code>boolean isEmpty()</code>	Returns true if the invoking collection is empty. Otherwise, returns false .
<code>Iterator<E> iterator()</code>	Returns an iterator for the invoking collection.
<code>boolean remove(Object obj)</code>	Removes one instance of <i>obj</i> from the invoking collection. Returns true if the element was removed. Otherwise, returns false .
<code>boolean removeAll(Collection<?> c)</code>	Removes all elements of <i>c</i> from the invoking collection. Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false .
<code>boolean retainAll(Collection<?> c)</code>	Removes all elements from the invoking collection except those in <i>c</i> . Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false .
<code>int size()</code>	Returns the number of elements held in the invoking collection.
<code>Object[] toArray()</code>	Returns an array that contains all the elements stored in the invoking collection. The array elements are copies of the collection elements.
<code><T> T[] toArray(T array[])</code>	Returns an array that contains the elements of the invoking collection. The array elements are copies of the collection elements. If the size of <i>array</i> equals the number of elements, these are returned in <i>array</i> . If the size of <i>array</i> is less than the number of elements, a new array of the necessary size is allocated and returned. If the size of <i>array</i> is greater than the number of elements, the array element following the last collection element is set to null . An ArrayStoreException is thrown if any collection element has a type that is not a subtype of <i>array</i> .

TABLE 17-1 The Methods Defined by **Collection**

Core Methods of Collection Interface- It defines most common methods which are applicable for any object.

1. **add(E e):** Adds the specified element to the collection.

Example:

```
Collection<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10); // Adds 10 to the collection
```

```
System.out.println(numbers); // Output: [10]
```

addAll(Collection<? extends E> c): Adds all elements from the specified collection to the current collection.

Example:

```
Collection<Integer> numbers1 = new ArrayList<>();
```

```
numbers1.add(10);
```

```
numbers1.add(20);
```

```
Collection<Integer> numbers2 = new ArrayList<>();
```

```
numbers2.add(30);
```

```
numbers2.add(40);
```

```
numbers1.addAll(numbers2);
```

```
System.out.println(numbers1); // Output: [10, 20, 30, 40]
```

remove(Object o):

- Removes the specified element from the collection.
- **Example:**

```
Collection<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10);
```

```
numbers.add(20);
```

```
numbers.remove(10); // Removes the element 10
```

```
System.out.println(numbers); // Output: [20]
```

removeAll(Collection<?> c): Removes all elements from the current collection that are also in the specified collection.

Example:

```
Collection<Integer> numbers1 = new ArrayList<>();
```

```
numbers1.add(10);
```

```
numbers1.add(20);
```

```
numbers1.add(30);
```

```
Collection<Integer> numbers2 = new ArrayList<>();
```

```
numbers2.add(20);
```

```
numbers2.add(30);
```

```
numbers1.removeAll(numbers2);
```

```
System.out.println(numbers1); // Output: [10]
```

clear(): Removes all elements from the collection, making it empty.

Example:

```
Collection<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10);
```

```
numbers.add(20);
```

```
numbers.clear(); // Removes all elements
```

```
System.out.println(numbers); // Output: [ ]
```

retainAll(Collection<?> c): Retains only the elements in the current collection that are also present in the specified collection.

Example:

```
Collection<Integer> numbers1 = new ArrayList<>();
```

```
numbers1.add(10);
```

```
numbers1.add(20);
```

```
numbers1.add(30);
```

```
Collection<Integer> numbers2 = new ArrayList<>();
```

```
numbers2.add(20);
```

```
numbers2.add(30);
```

```
numbers1.retainAll(numbers2);
```

```
System.out.println(numbers1); // Output:  
[20, 30]
```

Except the specific collection remove
all remaining objects

isEmpty():

- Checks if the collection is empty (i.e., has no elements).

Example:

```
Collection<Integer> numbers = new ArrayList<>();
```

```
System.out.println(numbers.isEmpty()); // Output: true
```

```
numbers.add(10);
```

```
System.out.println(numbers.isEmpty()); // Output: false
```

size():

- Returns the number of elements in the collection.

Example:

```
Collection<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10);
```

```
numbers.add(20);
```

```
System.out.println(numbers.size()); // Output: 2
```

contains(Object o):

- Checks if the specified element is present in the collection.
- **Example:**

```
Collection<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10);
```

```
numbers.add(20);
```

```
System.out.println(numbers.contains(10)); // Output: true
```

```
System.out.println(numbers.contains(30)); // Output: false
```

containsAll(Collection<?> c): Checks if the current collection contains all the elements of the specified collection.

Example:

```
Collection<Integer> numbers1 = new ArrayList<>();
```

```
numbers1.add(10);
```

```
numbers1.add(20);
```

```
numbers1.add(30);
```

```
System.out.println(numbers1.containsAll(numbers2));  
// Output: true
```

```
Collection<Integer> numbers2 = new ArrayList<>();
```

```
numbers2.add(20);
```

```
numbers2.add(30);
```

toArray(): collection to array

- Returns an array containing all elements in the collection.

Example:

```
Collection<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10);
```

```
numbers.add(20);
```

```
Object[] array = numbers.toArray();
```

```
System.out.println(Arrays.toString(array)); // Output: [10, 20]
```

iterator() method in Collections:

The **iterator()** method returns an iterator over the elements in the collection. Iterators are used to loop through a collection's elements one by one. It's a common way to access elements, especially when you want to avoid using an index or when dealing with collections that don't support direct indexing.

An iterator allows the programmer to iterate over elements in a collection in a sequential manner.

```
ArrayList<Integer> list = new ArrayList<>();  
    list.add(1);  
    list.add(2);  
    list.add(3);  
    list.add(4);  
  
    // Get an iterator for the list  
    Iterator<Integer> iterator = list.iterator();  
  
    // Iterate through the collection using the iterator  
    while (iterator.hasNext()) {  
        Integer element = iterator.next();  
        System.out.println(element);  
    }
```

equals() method in Collections:

The equals() method is used to compare two collections for equality.

```
ArrayList<Integer> list1 = new ArrayList<>();
```

```
list1.add(1);
```

```
list1.add(2);
```

```
list1.add(3);
```

```
ArrayList<Integer> list2 = new ArrayList<>();
```

```
list2.add(1);
```

```
list2.add(2);
```

```
list2.add(3);
```

```
// Comparing the two collections using equals()
```

```
System.out.println("Are list1 and list2 equal? " +  
list1.equals(list2));
```

```
import java.util.*;

public class CollectionDemo {

    public static void main(String[] args) {

        // Create Collection

        Collection<Integer> c1 = new ArrayList<>();

        Collection<Integer> c2 = new ArrayList<>();

        c1.add(10);      // add()

        c1.add(20);

        c1.add(30);

        System.out.println("c1 after add(): " + c1);
```

```
c2.add(40);           // addAll()
```

```
c2.add(50);
```

```
c1.addAll(c2);
```

```
System.out.println("c1 after addAll(): " + c1);
```

```
System.out.println("Contains 20? " + c1.contains(20));           // contains()
```

```
System.out.println("Contains all elements of c2? " + c1.containsAll(c2));
```

```
// containsAll()
```

```
System.out.println("Is c1 empty? " + c1.isEmpty());           // isEmpty()
```

```
System.out.println("Size of c1: " + c1.size());           // size()
```

```
System.out.print("Using iterator(): ");
```

```
// iterator()
```

```
Iterator<Integer> it = c1.iterator();
```

```
while (it.hasNext()) {
```

```
    System.out.print(it.next() + " ");
```

```
}
```

```
System.out.println();
```

```
c1.remove(20);
```

```
// remove()
```

```
System.out.println("After remove(20): " + c1);
```

```
c1.removeAll(c2);
```

```
// removeAll()
```

```
System.out.println("After removeAll(c2): " + c1);
```

c1.add(100); **//retainAll()**

c1.add(200);

c2.add(100);

c1.retainAll(c2);

System.out.println("After retainAll(c2): " + c1);

System.out.println("c1 equals c2? " + c1.equals(c2)); **// equals()**

System.out.println("HashCode of c1: " + c1.hashCode()); **// hashCode()**

Object[] arr = c1.toArray(); **// toArray()**

System.out.println("Array using toArray(): " + Arrays.toString(arr));

```
// toArray(T[] a)
```

```
Integer[] arr2 = c1.toArray(new Integer[0]);
```

```
System.out.println("Typed Array: " + Arrays.toString(arr2));
```

```
// clear()
```

```
c1.clear();
```

```
System.out.println("After clear(): " + c1);
```

```
// final isEmpty check
```

```
System.out.println("Is c1 empty now? " + c1.isEmpty());
```

```
}
```

```
}
```

c1 after add(): [10, 20, 30]

c1 after addAll(): [10, 20, 30, 40, 50]

Contains 20? true

Contains all elements of c2? true

Is c1 empty? false

Size of c1: 5

Using iterator(): 10 20 30 40 50

After remove(20): [10, 30, 40, 50]

After removeAll(c2): [10, 30]

After retainAll(c2): [100]

c1 equals c2? false

HashCode of c1: 100

Array using toArray(): [100]

Typed Array: [100]

After clear(): []

Is c1 empty now? true

Output

ii. List :

- * List is child interface of Collection.
- * If we want to represent a group of individual objects as a single entity where duplicates are allowed and insertion order preserved then we should go for List.

The List Interface in Java

The List interface in Java is part of the Java Collections Framework and extends the Collection interface. It represents a collection that stores a sequence of elements. Unlike other types of collections (like sets), a list allows duplicate elements, and elements can be accessed or inserted at specific positions using a zero-based index. The List interface provides a way to maintain an ordered collection where you can **insert, remove, or access elements by their index.**

Syntax

The List interface is a **generic interface**, meaning you specify the type of elements it will hold:

```
interface List<E>
```

Here, E represents the type of elements the list will contain (e.g., Integer, String, etc.).

Common Methods Defined by **List**

Here are some important methods defined by the **List** interface:

- **add(E e)**: Adds an element to the list (at the end).
- **add(int index, E element)**: Inserts the specified element at the specified position in the list.
- **addAll(Collection<? extends E> c)**: Adds all elements from the specified collection to the end of the list.
- **addAll(int index, Collection<? extends E> c)**: Adds all elements from the specified collection at the specified index.
- **get(int index)**: Returns the element at the specified index.

set(int index, E element): Replaces the element at the specified position with the specified element.

remove(int index): Removes the element at the specified index.

indexOf(Object o): Returns the index of the first occurrence of the specified element in the list.

lastIndexOf(Object o): Returns the index of the last occurrence of the specified element.

subList(int fromIndex, int toIndex): Returns a sublist of the list between the specified **fromIndex** and **toIndex** (excluding **toIndex**).

size(): Returns the number of elements in the list.

```
import java.util.ArrayList;

import java.util.List;

public class ListExample {

    public static void main(String[] args) {

        // Creating a List of Strings

        List<String> fruits = new ArrayList<>();

        // Adding elements to the list using add()

        fruits.add("Apple");

        fruits.add("Banana");

        fruits.add("Orange");

        fruits.add("Apple"); // List allows duplicates
    }
}
```

```
System.out.println("List after adding elements: " + fruits);
```

```
// Adding an element at a specific index using add(int index, E element)
```

```
fruits.add(1, "Grapes"); // Adds Grapes at index 1
```

```
System.out.println("List after adding an element at index 1: " + fruits);
```

```
// Accessing an element using get()
```

```
String fruitAtIndex2 = fruits.get(2); // Gets the element at index 2
```

```
System.out.println("Element at index 2: " + fruitAtIndex2);
```

```
// Modifying an element using set()
```

```
fruits.set(3, "Mango"); // Replaces the element at index 3 with Mango
```

```
System.out.println("List after modifying the element at index 3: " + fruits);
```

// Finding the index of an element using indexOf()

```
int indexOfApple = fruits.indexOf("Apple");
```

```
System.out.println("Index of 'Apple': " + indexOfApple);
```

// Finding the last index of an element using lastIndexOf()

```
int lastIndexOfApple = fruits.lastIndexOf("Apple");
```

```
System.out.println("Last index of 'Apple': " + lastIndexOfApple);
```

// Getting a sublist using subList()

```
List<String> sublist = fruits.subList(1, 4); // Gets elements from index 1 to 3  
(exclusive of index 4)
```

```
System.out.println("Sublist from index 1 to 4: " + sublist);
```

// Removing an element at a specific index using remove()

fruits.remove(2); // Removes the element at index 2

System.out.println("List after removing the element at index 2: " + fruits);

// Size of the list

System.out.println("Size of the list: " + fruits.size());

}

}

Output:

List after adding elements: [Apple, Banana, Orange, Apple]

List after adding an element at index 1: [Apple, Grapes, Banana, Orange, Apple]

Element at index 2: Banana

List after modifying the element at index 3: [Apple, Grapes, Banana, Mango, Apple]

Index of 'Apple': 0

Last index of 'Apple': 4

Sublist from index 1 to 4: [Grapes, Banana, Mango]

List after removing the element at index 2: [Apple, Grapes, Mango, Apple]

Size of the list: 4

Important Notes:

- **Duplicates:** Lists allow duplicate elements, so we can have the same element more than once in a list (e.g., "Apple" appears twice in the list).
- **Index-based access:** You can add, modify, or access elements at specific positions using their indices, which makes lists more flexible for ordered collections.
- **Exceptions:** As mentioned in the original explanation, if you use an invalid index, like `get(10)` on a list with fewer than 10 elements, you will get an `IndexOutOfBoundsException`. Similarly, if the list does not allow null values, a `NullPointerException` may be thrown if null is added.

ArrayList<>(): The ArrayList class is a **concrete implementation** of the List interface. It dynamically resizes as elements are added or removed. The empty angle brackets <> are used to specify the type of elements that the ArrayList will hold, which, in this case, will be String elements (since the List<String> reference was declared earlier).

- ArrayList is part of Java's **java.util** package, and it implements the List interface, which means it provides all the functionalities defined by the List interface.
- The <> is the **diamond operator**, which is used in Java 7 and later to allow the compiler to infer the type (you don't need to explicitly repeat <String> on the right side since the compiler knows it from the left side). This is a shorthand for the explicit new ArrayList<String>().

The Collection Classes

ArrayList:

- It's a **resizable array** in Java — like an array that can grow and shrink automatically.
- Part of `java.util` package and implements the `List` interface.
- Stores objects **in insertion order** and allows duplicates.

Even though `ArrayList` resizes automatically:

- **`ensureCapacity(int capacity)`** boosts performance if you know you'll add a lot of elements.
- **`trimToSize()`** reduces memory usage when you've added and removed items and want to shrink it to fit exactly.

```
import java.util.ArrayList;
```

```
public class CapacityDemo {
```

```
    public static void main(String[] args) {
```

```
        // Create an ArrayList with initial capacity 5
```

```
        ArrayList<String> list = new ArrayList<>(5);
```

```
        // Ensuring the capacity is at least 100 (for performance)
```

```
        list.ensureCapacity(100);
```

// Adding some elements (not 100, just a few)

```
list.add("Java");
```

```
list.add("Python");
```

```
list.add("C++");
```

```
System.out.println("List before trimming: " + list);
```

// Now let's trim the internal array size to match current list size (3 elements)

```
list.trimToSize();
```

// List still contains the same items, but memory is freed

```
System.out.println("List after trimming: " + list);
```

```
}
```

```
}
```

Obtaining an Array from an ArrayList

When working with ArrayList, you will sometimes want to obtain an actual array that contains the contents of the list. You can do this by calling `toArray( )`, which is defined by Collection.

Several reasons exist why you might want to convert a collection into an array, such as:

- To obtain faster processing times for certain operations
- To pass an array to a method that is not overloaded to accept a collection
- To integrate collection-based code with legacy code that does not understand collections

there are two versions of `toArray( )`, which are shown again here for your convenience:

`Object[ ] toArray( )`

`<T> T[ ] toArray(T array[ ])`

The first returns an array of `Object`. The second returns an array of elements that have the same type as `T`. Normally, the second form is more convenient because it returns the proper type of array.

```
import java.util.*;
```

```
class ObjectArrayExample {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<String> list = new ArrayList<>();
```

```
        list.add("Apple");
```

```
        list.add("Banana");
```

```
        list.add("Cherry");
```

```
        // Convert to Object[]
```

```
        Object[] objArray = list.toArray();
```

Object[] toArray()

// Display elements by casting to String

```
System.out.println("Using Object[ ] toArray():");
```

```
for (Object obj : objArray) {
```

```
    String fruit = (String) obj; //explicit casting is needed
```

```
    System.out.println(fruit);
```

```
}
```

```
}
```

```
}
```

```
import java.util.*;

class ArrayListToArray {

    public static void main(String args[]) {

        // Create an array list.

        ArrayList<Integer> al = new ArrayList<Integer>();

        // Add elements to the array list.

        al.add(1);
        al.add(2);
        al.add(3);
        al.add(4);

        System.out.println("Contents of al: " + al);
```

// Get the array.

```
Integer ia[] = new Integer[al.size()];
```

```
ia = al.toArray(ia);
```

```
int sum = 0;
```

// Sum the array.

```
for(int i : ia) sum += i;
```

```
System.out.println("Sum is: " + sum);
```

```
}
```

```
}
```

The output from the program is shown here:

Contents of al: [1, 2, 3, 4]

Sum is: 10

The program begins by creating a **collection of integers**. Next, **toArray()** is called and it obtains **an array of Integers**. Then, the contents of that array are summed by use of a for-each style for loop.

The LinkedList Class

The LinkedList class **extends** **AbstractSequentialList** and **implements** the **List, Deque, and Queue interfaces**. It provides a linked-list data structure. LinkedList is a generic class that has this declaration:

class LinkedList<E>

Here, E specifies the type of objects that the list will hold. LinkedList has the **two constructors** shown here:

LinkedList()

LinkedList(Collection<? extends E> c)

The first constructor builds an empty linked list. The second constructor builds a linked list that is initialized with the elements of the collection c.

LinkedList Implements:

- **List** → allows **indexed access**, like add(index, element), get(index), set(index, value).
- **Deque (Double-Ended Queue)** → allows operations at **both ends**, like addFirst(), addLast(), removeFirst(), etc.

```
import java.util.*;
```

```
class LinkedListExample {  
    public static void main(String[] args) {  
        LinkedList<String> ll = new LinkedList<>();  
  
        // Add elements to the end (as List or addLast())  
  
        ll.add("F");  
  
        ll.add("B");  
    }  
}
```

```
ll.add("D");
```

```
ll.add("E");
```

```
ll.add("C");
```

// Add element to the beginning using addFirst()

```
ll.addFirst("A");
```

// Add element to the end using addLast()

```
ll.addLast("Z");
```

// Add element at specific index using add(index, element)

```
ll.add(1, "A2");
```

```
System.out.println("Original contents of ll: " + ll); // Output: [A, A2, F, B, D, E, C, Z]
```

```
ll.remove("F"); // Remove by value (first occurrence of "F")
```

```
// Remove by index (removes the element at index 2)
```

```
ll.remove(2);
```

```
System.out.println("Contents of ll after deletion: " + ll);
```

```
// Output: [A, A2, D, E, C, Z]
```

```
// Remove first and last elements
```

```
ll.removeFirst(); // Removes "A"
```

```
ll.removeLast();
```

```
System.out.println("ll after deleting first and last: " + ll);
```

```
// Output: [A2, D, E, C]
```

```
// Get element at index 2
```

```
String val = ll.get(2); // val = "E"
```

```
// Modify element at index 2
```

```
ll.set(2, val + " Changed");
```

```
System.out.println("ll after change: " + ll);
```

```
// Output: [A2, D, E Changed, C]
```

```
}
```

```
}
```

Method	Purpose
add(E)	Adds to the end of the list
addFirst(E)	Adds to the beginning of the list
addLast(E)	Adds to the end of the list
add(index, E)	Inserts at specific index
remove(Object)	Removes first occurrence of the object
remove(index)	Removes element at a specific index
removeFirst()	Removes the first element
removeLast()	Removes the last element
get(index)	Gets element at the index
set(index, E)	Replaces element at index with new value

Method	Description
<code>void add(int <i>index</i>, E <i>obj</i>)</code>	Inserts <i>obj</i> into the invoking list at the index passed in <i>index</i> . Any preexisting elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten.
<code>boolean addAll(int <i>index</i>, Collection<? extends E> <i>c</i>)</code>	Inserts all elements of <i>c</i> into the invoking list at the index passed in <i>index</i> . Any preexisting elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten. Returns true if the invoking list changes and returns false otherwise.
<code>static <E> List<E> copyOf(Collection<? extends E> <i>from</i>)</code>	Returns a list that contains the same elements as that specified by <i>from</i> . The returned list is unmodifiable and value-based. Null values are not allowed.
<code>E get(int <i>index</i>)</code>	Returns the object stored at the specified index within the invoking collection.
<code>int indexOf(Object <i>obj</i>)</code>	Returns the index of the first instance of <i>obj</i> in the invoking list. If <i>obj</i> is not an element of the list, -1 is returned.

int lastIndexOf(Object <i>obj</i>)	Returns the index of the last instance of <i>obj</i> in the invoking list. If <i>obj</i> is not an element of the list, -1 is returned.
ListIterator<E> listIterator()	Returns an iterator to the start of the invoking list.
ListIterator<E> listIterator(int <i>index</i>)	Returns an iterator to the invoking list that begins at the specified <i>index</i> .
static <E> List<E> of(<i>parameter-list</i>)	Creates an unmodifiable value-based list containing the elements specified in <i>parameter-list</i> . Null elements are not allowed. Many overloaded versions are provided. See the discussion in the text for details.
E remove(int <i>index</i>)	Removes the element at position <i>index</i> from the invoking list and returns the deleted element. The resulting list is compacted. That is, the indexes of subsequent elements are decremented by one.
default void replaceAll(UnaryOperator<E> <i>opToApply</i>)	Updates each element in the list with the value obtained from the <i>opToApply</i> function.
E set(int <i>index</i> , E <i>obj</i>)	Assigns <i>obj</i> to the location specified by <i>index</i> within the invoking list. Returns the old value.

default void	Sorts the list using the comparator specified by <i>comp</i> .
sort(Comparator<? super E> <i>comp</i>)	
List<E> subList(int <i>start</i> , int <i>end</i>)	Returns a list that includes elements from <i>start</i> to <i>end</i> -1 in the invoking list. Elements in the returned list are also referenced by the invoking object.

Table 20-3 The Methods Declared by **List**

Accessing a Collection via an Iterator

What is an Iterator?

An **Iterator** is an object that allows you to traverse a collection one element at a time, **without exposing the underlying structure** (like array, linked list, etc.).

Basic Methods in Iterator<E>:

Method	Description
boolean hasNext()	Returns true if there are more elements
E next()	Returns the next element
void remove()	Removes the current element (optional operation)

Method	Description
boolean hasNext()	Returns true if there are more elements. Otherwise, returns false .
E next()	Returns the next element. Throws NoSuchElementException if there is not a next element.
void remove()	Removes the current element. Throws IllegalStateException if an attempt is made to call remove() that is not preceded by a call to next() .

TABLE 17-8 The Methods Defined by **Iterator**

Using an Iterator

In general, to use an iterator to cycle through the contents of a collection, follow these steps:

1. Obtain an iterator to the start of the collection by calling the collection's `iterator()` method.
2. Set up a loop that makes a call to `hasNext()`. Have the loop iterate as long as `hasNext()` returns true.
3. Within the loop, obtain each element by calling `next()`.

```
import java.util.*;

public class IteratorDemo {

    public static void main(String[] args) {

        // Step 1: Create and populate a collection

        ArrayList<String> fruits = new ArrayList<>();

        fruits.add("Apple");

        fruits.add("Banana");

        fruits.add("Mango");

        // Step 2: Obtain the iterator

        Iterator<String> itr = fruits.iterator();
    }
}
```

// Step 3: Loop through the collection

```
System.out.println("Fruits list using Iterator:");  
while (itr.hasNext()) {  
    String fruit = itr.next();  
    System.out.println(fruit);  
}  
}  
}
```

What is a ListIterator?

ListIterator extends Iterator and is used specifically with **List** implementations (like ArrayList, LinkedList). It allows:

- Forward and backward traversal
- Modification of list during iteration

Additional Methods in ListIterator<E>:

Method	Description
boolean hasPrevious()	Checks if there are elements before
E previous()	Gets the previous element
int nextIndex()	Index of the next element
int previousIndex()	Index of the previous element
void set(E e)	Replaces last returned element with e
void add(E e)	Adds an element before the next element

Method	Description
<code>void add(E obj)</code>	Inserts <i>obj</i> into the list in front of the element that will be returned by the next call to next() .
<code>boolean hasNext()</code>	Returns true if there is a next element. Otherwise, returns false .
<code>boolean hasPrevious()</code>	Returns true if there is a previous element. Otherwise, returns false .
<code>E next()</code>	Returns the next element. A NoSuchElementException is thrown if there is not a next element.
<code>int nextIndex()</code>	Returns the index of the next element. If there is not a next element, returns the size of the list.
<code>E previous()</code>	Returns the previous element. A NoSuchElementException is thrown if there is not a previous element.
<code>int previousIndex()</code>	Returns the index of the previous element. If there is not a previous element, returns -1.
<code>void remove()</code>	Removes the current element from the list. An IllegalStateException is thrown if remove() is called before next() or previous() is invoked.
<code>void set(E obj)</code>	Assigns <i>obj</i> to the current element. This is the element last returned by a call to either next() or previous() .

TABLE 17-9 The Methods Defined by **ListIterator**

```
import java.util.*;
```

```
public class ListIteratorDemo {
```

```
    public static void main(String[] args) {
```

```
        LinkedList<String> colors = new LinkedList<>();
```

```
        colors.add("Red");
```

```
        colors.add("Green");
```

```
        colors.add("Blue");
```

```
        // Obtain the ListIterator
```

```
        ListIterator<String> listItr = colors.listIterator();
```

ListIterator

```
System.out.println("Forward traversal:");  
    while (listItr.hasNext()) {  
        System.out.println(listItr.next());  
    }  
    System.out.println("Backward traversal:");  
    while (listItr.hasPrevious()) {  
        System.out.println(listItr.previous());  
    }  
}  
}
```

Use Case	Iterator	ListIterator
All collection types	✓	✗ Only for Lists
Forward traversal only	✓	✓
Backward traversal	✗	✓
Modify list while iterating	✗	✓ (use <code>set()</code> / <code>add()</code> / <code>remove()</code> safely)

The For-Each Alternative to Iterators

The **enhanced for loop** allows you to iterate over collections and arrays **more simply** than using an explicit Iterator.

```
for (Type var : collection) {  
    // use var  
}
```

When to use it?

- When you want to **read** elements (not modify or remove).
- When you don't care about the **index**.
- When you only need **forward traversal**.
- When working with any Iterable (like ArrayList, HashSet, etc.).

```
import java.util.*;

public class ForEachDemo {

    public static void main(String[] args) {

        // Create an ArrayList of integers

        ArrayList<Integer> vals = new ArrayList<>();

        // Add elements to the list

        vals.add(1);
        vals.add(2);
        vals.add(3);
        vals.add(4);
        vals.add(5);
```

// Display the values using for-each

```
System.out.print("Original contents of vals: ");
```

```
for (int v : vals) {
```

```
System.out.print(v + " ");
```

```
}
```

```
System.out.println();
```

```
int sum = 0;
```

```
for (int v : vals) {
```

```
sum += v;
```

```
}
```

```
System.out.println("Sum of values: " + sum);
```

```
}
```

Output:

Original contents of vals: 1 2 3 4 5

Sum of values: 15

```
}
```

Limitations of For-Each Loop

Limitation	Why?	Alternative
Can't remove elements	Iterator required	Use <code>Iterator.remove()</code>
Can't update element values	No access to index	Use regular for with <code>list.set()</code>
Can't go in reverse order	Not supported	Use <code>ListIterator</code>

```
for (int v : vals) {
```

```
    if (v == 3)
```

```
        vals.remove(v); //  This throws ConcurrentModificationException
```

```
}
```

Splititerator in Java (JDK 8)- Defined by Splititerator interface

A **Splititerator** is an advanced iterator introduced in **JDK 8**.

It is used to **traverse (iterate) elements** of a collection — but with extra power, especially for **parallel processing**.

Like Iterator, it goes through elements

But unlike Iterator, it can:

- **Split data into parts** (for parallel processing)
- Combine hasNext() + next() into one method → tryAdvance()

Declaration

interface Splititerator<T>

- T → type of elements

Important Methods (Core)

1. **tryAdvance()**

boolean tryAdvance(Consumer<? super T> action)

Processes **one element at a time**

Returns **true** if element exists

Consumer from `java.util.function`

- A functional interface
- Represents: “**Take input and do something (no return)**”

2. **forEachRemaining()**

void forEachRemaining(Consumer<? super T> action)

Processes **all remaining elements**

void accept(T t);

3. **trySplit()**

Splititerator<T> trySplit()

Splits the data into two parts

Used for **parallel processing**

4. **estimateSize()**

long estimateSize()

Returns estimated number of elements

5. **characteristics()**

int characteristics()

Describes properties like:

- ORDERED
- DISTINCT
- SORTED

```
import java.util.*;

class SplitIteratorDemo {

    public static void main(String[] args) {

        ArrayList<Double> vals = new ArrayList<>();  // Create an ArrayList for doubles

        vals.add(1.0);                                // Add values to the list
        vals.add(2.0);
        vals.add(3.0);
        vals.add(4.0);
        vals.add(5.0);

        System.out.println("Contents of vals:");
```

`Splititerator<Double> splitr = vals.splititerator();` **// Use tryAdvance() to display contents of vals**

```
while (splitr.tryAdvance((n) -> System.out.println(n)));
```

```
System.out.println();
```

// Create new list that contains square roots

```
splitr = vals.splititerator(); // reset spliterator
```

```
ArrayList<Double> sqrs = new ArrayList<>();
```

```
while (splitr.tryAdvance((n) -> sqrs.add(Math.sqrt(n))));
```

// Use forEachRemaining() to display contents of sqrs

```
System.out.println("Contents of sqrs:");
```

```
splitr = sqrs.splitIterator();  
    splitr.forEachRemaining((n) -> System.out.println(n));  
    System.out.println();  
}  
}
```

Contents of vals:

1.0

2.0

3.0

4.0

5.0

Contents of sqrs:

1.0

1.4142135623730951

1.7320508075688772

2.0

2.23606797749979

Output:

Feature	Iterator	Splitterator
Traversal	Sequential	Sequential + Parallel
Methods	hasNext (), next ()	tryAdvance ()
Splitting	No	Yes (trySplit ())
Performance	Normal	Better for large data

When to Use Splitter?

Use it when:

- Working with **large datasets**
- Need **parallel processing**
- Using **Streams API internally**

```
import java.util.*;

class ParallelExample {

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1,2,3,4,5,6);

        Spliterator<Integer> sp1 = list.spliterator();

        Spliterator<Integer> sp2 = sp1.trySplit(); // splitting

        new Thread(() -> {                                     // First half

            sp1.forEachRemaining(n -> System.out.println("Thread1: " + n));

        }).start();
```

```
new Thread(() -> {                                     // Second half
    sp2.forEachRemaining(n -> System.out.println("Thread2: " + n));
}).start();
}
}
```

Output:

Thread1: 4

Thread2: 1

Thread1: 5

Thread2: 2

Thread1: 6

Thread2: 3

Storing User-Defined Classes in Collections

- For the sake of simplicity, the foregoing examples have stored built-in objects, such as **String** or **Integer**, in a collection.
- Of course, collections are not limited to the storage of built-in objects.
- The power of collections is that they can **store any type of object**, including objects of classes that you create.
 - For example, **consider the following example that uses a `LinkedList` to store mailing addresses:**

```
import java.util.*;  
class Address {  
    private String name;  
    private String street;  
    private String city;  
    private String state;  
    private String code;
```

```
    Address(String n, String s, String c,  
            String st, String cd) {  
        name = n;  
        street = s;  
        city = c;  
        state = st;  
        code = cd;  
    }  
}
```

```
public String toString() {  
    return name + "\n" + street + "\n" + city + " " + state + " " + code;  
}  
}
```

```
class MailList {  
  
    public static void main(String args[]) {  
  
        LinkedList<Address> ml = new LinkedList<Address>();  
  
        // Add elements to the linked list.  
  
        ml.add(new Address("J.W. West", "11 Oak Ave", "Urbana", "IL", "61801"));  
    }  
}
```

```
ml.add(new Address("Ralph Baker", "1142 Maple Lane", "Mahomet", "IL",  
"61853"));
```

```
ml.add(new Address("Tom Carlton", "867 Elm St", "Champaign", "IL",  
"61820"));
```

```
// Display the mailing list.
```

```
for(Address element : ml)
```

```
System.out.println(element + "\n");
```

```
System.out.println();
```

```
}
```

```
}
```

The output from the program is shown here:

J.W. West

11 Oak Ave

Urbana IL 61801

Output

Ralph Baker

1142 Maple Lane

Mahomet IL 61853

Tom Carlton

867 Elm St

Champaign IL 61820

Feature

Generic collections

No boilerplate data handling

Iterable support

toString() integration

Minimal lines of code

Benefit

Store any object — built-in or custom

No need to code custom list or resize logic

Easily loop over any type with for-each

Cleaner output and better debugging

Focus on logic, not plumbing

The Set Interface in Java

The Set interface in Java is part of the **Java Collections Framework** and extends the **Collection interface**. It defines the behavior of a collection that **does not allow duplicate elements**. This means a Set can contain each element only once, and if an attempt is made to **add a duplicate element**, the **add() method will return false** (indicating that the element was not added).

A Set does not provide any additional methods beyond those inherited from Collection, but its unique characteristic is that it does not allow duplicates.

Syntax of the Set Interface:

The Set interface is a **generic interface**, meaning you specify the type of elements it will hold (e.g., Integer, String):

interface Set<E>

Common Methods in Set

Since the **Set interface extends Collection**, it inherits methods like `add(E e)`, `remove(Object o)`, and `size()`. Some important methods specific to Set include:

- **add(E e)**: Adds the specified element to the set. Returns false if the element is already present (no duplicates).
- **contains(Object o)**: Checks if the set contains the specified element.
- **remove(Object o)**: Removes the specified element from the set.
- **size()**: Returns the number of elements in the set.
- **clear()**: Removes all elements from the set.
- **isEmpty()**: Checks if the set is empty.

```
public class SetExample {  
    public static void main(String[] args) {  
        // Creating a Set of Strings  
        Set<String> fruits = new HashSet<>();  
        // Adding elements to the Set  
        fruits.add("Apple");  
        fruits.add("Banana");  
        fruits.add("Orange");  
        fruits.add("Apple"); // Duplicate element, won't be added  
        System.out.println("Set after adding elements: " + fruits);  
    }  
}
```

// Checking if an element is present in the Set

```
boolean containsApple = fruits.contains("Apple");  
System.out.println("Does the set contain 'Apple'? " + containsApple); //  
true
```

// Removing an element from the Set

```
fruits.remove("Banana");  
System.out.println("Set after removing 'Banana': " + fruits);  
// Checking the size of the Set  
System.out.println("Size of the set: " + fruits.size()); // 2
```

// Clearing the Set

```
fruits.clear();
```

```
System.out.println("Set after clearing: " + fruits); // Empty Set
```

```
}
```

```
}
```

SortedSet Interface

SortedSet is an interface in Java that:

- **Stores unique elements** (like Set)
- **Maintains elements in sorted (ascending) order**
- **Extends SequencedSet**, so it also supports ordered operations like first/last

Declaration

interface SortedSet<E>

- <E> → type of elements (Integer, String, etc.)

Method	Description
Comparator<? super E> comparator()	Returns the invoking sorted set's comparator. If the natural ordering is used for this set, null is returned.
E getFirst() / E first()	Returns the first element in the invoking sorted set.
SortedSet<E> headSet(E end)	Returns a SortedSet containing those elements less than <i>end</i> that are contained in the invoking sorted set. Elements in the returned sorted set are also referenced by the invoking sorted set.
default E void removeFirst()	Removes and returns the first element in the sorted set.
default E void removeLast()	Removes and returns the first element in the sorted set.
E getLast() / E last()	Returns the last element in the invoking sorted set.
SortedSet<E> subSet(E start, E end)	Returns a SortedSet that includes those elements between <i>start</i> and <i>end</i> -1. Elements in the returned collection are also referenced by the invoking object.
SortedSet<E> tailSet(E start)	Returns a SortedSet that contains those elements greater than or equal to <i>start</i> that are contained in the sorted set. Elements in the returned set are also referenced by the invoking object.

Table 20-4 Useful Methods Declared by **SortedSet**

1. comparator()

Comparator<? super E> comparator()

- Returns the **comparator used for sorting**
- If natural ordering is used → returns null

Example:

- Strings sorted alphabetically → null
- Custom sorting → returns comparator

2. first() / getFirst()

E first()- Returns the **smallest (first) element**

Example: [10, 20, 30] → first() = 10

3. last() / getLast()

E last()- Returns the **largest (last) element**

Example: [10, 20, 30] → last() = 30

4. headSet(E end) **SortedSet<E> headSet(E end)**

- Returns elements **less than end**

Example: Set: [10, 20, 30, 40] headSet(30) → [10, 20]

5. tailSet(E start) **SortedSet<E> tailSet(E start)**

- Returns elements **greater than or equal to start**

Example: tailSet(20) → [20, 30, 40]

6. subSet(E start, E end) **SortedSet<E> subSet(E start, E end)**

- Returns elements **from start to end - 1**

Example: subSet(20, 40) → [20, 30]

7. removeFirst() **E removeFirst()**

- Removes and returns the **smallest element**

8. removeLast() **E removeLast()**

- Removes and returns the **largest element**

```
import java.util.*;

public class TestComparator1 {
    public static void main(String[] args) {
        TreeSet<String> set = new TreeSet<>();
        set.add("Banana");
        set.add("Apple");
        set.add("Mango");
        System.out.println("Sorted Set: " + set);
        Comparator<? super String> comp = set.comparator();
        System.out.println("Comparator: " + comp);
    }
}
```

Output:

Sorted Set: [Apple, Banana,
Mango]
Comparator: null

```
Comparator<Object> comp = (o1, o2) -> ((String)o1).compareTo((String)o2);  
TreeSet<String> set = new TreeSet<>(comp);
```

This works because of:

```
Comparator<? super String>
```

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        SortedSet<Integer> set = new TreeSet<>();

        set.add(30);

        set.add(10);

        set.add(20);

        System.out.println(set);          // [10, 20, 30]

        System.out.println(set.first());  // 10

        System.out.println(set.last());   // 30

        System.out.println(set.headSet(20)); // [10]

    }

}
```

HashSet class

HashSet<E> is a class in Java that:

- **extends** AbstractSet
- **implements** the Set interface

It stores **unique elements only** (no duplicates)

Although we say:

“HashSet uses a hash table”

Internally, **HashSet actually uses a HashMap**

- Each element you add becomes a **key** in a hidden HashMap
- A dummy value (like a constant object) is stored

When you do: `set.add("Apple");`

Steps:

1. Java calls: `"Apple".hashCode()`
2. That hash code determines **where to store it**
3. If two elements have same hash → **collision handling**
(bucket/linked list/tree)

Key Features of HashSet

- ✗ No duplicates
- ✗ No guaranteed order
- ✓ Allows **one null element**
- ✓ Fast operations (due to hashing)

Example:

```
HashSet<String> set = new HashSet<>();
```

```
set.add("Apple");
```

```
set.add("Banana");
```

```
set.add("Apple"); // duplicate, ignored
```

```
System.out.println(set);
```

Output:

```
[Apple, Banana] // order not guaranteed
```

1. HashSet Constructors

① HashSet()

- Creates an empty HashSet
- **Default capacity = 16**
- **Default load factor = 0.75**

② HashSet(Collection<? extends E> c)

- Creates a HashSet and **adds all elements from collection c**
- Useful for converting List → Set

③ HashSet(int capacity)

- Sets initial size of internal table
- Helps avoid resizing if you know size in advance

④ HashSet(int capacity, float loadFactor)

Load Factor - Defines **when resizing happens**

Formula: Resize when: number of elements > capacity × loadFactor

Example:

- capacity = 16
- load factor = 0.75

$$16 \times 0.75 = 12$$

When elements exceed **12**, resizing occurs.

Requirement	Use
Insertion order	LinkedHashSet
Sorted order	TreeSet

```
import java.util.*;

class HashSetDemo {

    public static void main(String[] args) {

        HashSet<String> hs = new HashSet<String>();

        hs.add("Beta");
        hs.add("Alpha");
        hs.add("Eta");
        hs.add("Gamma");
        hs.add("Epsilon");
        hs.add("Omega");

        System.out.println(hs);    }    }
```

Output:

[Gamma, Eta, Alpha, Epsilon, Omega,
Beta]

- Order **may change** every time you run the program
- Because HashSet does **not maintain insertion order**

ArrayDeque Class

ArrayDeque is a resizable array implementation of the **Deque (Double-Ended Queue)** interface.

Key Points

- **Inheritance & Implementation**
 - Extends: AbstractCollection
 - Implements: Deque<E>

Generic Declaration: `class ArrayDeque<E>`

E represents the type of elements stored.

Important Features

1. Dynamic Resizing

- Internally uses a **dynamic array**.
- Automatically grows as elements are added.
- No fixed capacity limit.

2. Double-Ended Operations

- Supports insertion and deletion from **both ends**:
 - Front (head)
 - Rear (tail)

3. No Capacity Restrictions

- Unlike some queue implementations, it is **unbounded** (limited only by memory).

4. Faster than Stack and LinkedList

- More efficient than:
 - Stack (no synchronization overhead)
 - LinkedList (better cache performance)

5. Not Thread-Safe

- Not synchronized → use external synchronization if needed.

6. No Null Elements Allowed

- Inserting null will throw a **NullPointerException**.

ArrayDeque defines the following constructors:

1. ArrayDeque()

```
ArrayDeque<String> dq = new ArrayDeque<>();
```

- Creates an **empty deque**
- Default **initial capacity = 16**
- It **automatically grows** when needed

Important:

- You don't see resizing, but internally it doubles capacity when full.

2. ArrayDeque(int size)

```
ArrayDeque<String> dq = new ArrayDeque<>(50);
```

- Creates an empty deque with **initial capacity = 50**
- Useful when you already know approximate size → avoids resizing overhead

But:

- Still **dynamic** → grows beyond 50 if required

3. ArrayDeque(Collection<? extends E> c)

```
List<String> list = Arrays.asList("A", "B", "C");
```

```
ArrayDeque<String> dq = new ArrayDeque<>(list);
```

- Creates a deque **initialized with elements from another collection**
- Elements are inserted **in the same order**

```
import java.util.*;
class ArrayDequeDemo {
    public static void main(String[] args) {
        ArrayDeque<String> adq = new ArrayDeque<String>();
        // Use ArrayDeque like a stack (LIFO)
        adq.push("A");
        adq.push("B");
        adq.push("D");
        adq.push("E");
        adq.push("F");
```

**The following program
demonstrates ArrayDeque by
using it to create a stack:**

// Display elements by popping

```
System.out.print("Popping the stack: ");
```

```
while(adq.peek() != null) {
```

```
    System.out.print(adq.pop() + " ");
```

```
}
```

```
System.out.println();
```

```
}
```

```
}
```

Output

Popping the stack: F E D B A

Even though ArrayDeque is a **deque**, it can behave as:

- **Stack** → using push() and pop() (LIFO)
- **Queue** → using offer() and poll() (FIFO)

```
import java.util.*;

class ArrayDequeDemo {

    public static void main(String[] args) {

        ArrayDeque<String> adq = new ArrayDeque<String>();

        // Use ArrayDeque like a queue (FIFO)

        adq.offer("A");
        adq.offer("B");
        adq.offer("D");
        adq.offer("E");
        adq.offer("F");
```

**The following program
demonstrates ArrayDeque by
using it to create a queue:**

// Display elements by popping

```
System.out.print("Popping the stack: ");
```

```
while(adq.peek() != null) {
```

```
    System.out.print(adq.poll() + " ");
```

```
}
```

```
System.out.println();
```

```
}
```

```
}
```

Output

Popping the stack: A B D E F

Working with Maps

A **Map** stores data in **key–value pairs**.

Think of it like a dictionary:

- **Key** → unique identifier
- **Value** → data linked to that key

Example:

RollNo → Name

101 → abc

Key Characteristics

- **Keys are unique**
- **Values can be duplicated**
- **Stores data as (key, value) pairs**
- **Does not implement Iterable**
 - So, you **cannot directly use for-each loop on Map**
 - But you can use:
 - `keySet()`
 - `entrySet()`
 - `values()`

This will NOT work:

```
Map<String, Integer> map = new HashMap<>();
```

```
for (String key : map) { // ERROR  
    System.out.println(key);  
}
```

Reason: Map does not implement Iterable.

Correct ways to use for-each with Map

1. Iterate over keys

```
for (String key : map.keySet())  
{  
    System.out.println(key);  
}
```

2. Iterate over values

```
for (Integer value : map.values())  
{  
    System.out.println(value);  
}
```

3. Iterate over key-value pairs (BEST & most used)

```
for (Map.Entry<String, Integer> entry : map.entrySet())  
{  
    System.out.println(entry.getKey() + " " + entry.getValue());  
}
```

The Map Interfaces

Interface	Description
Map	Maps unique keys to values.
Map.Entry	Describes an element (a key/value pair) in a map. This is a nested interface of Map .
NavigableMap	Extends SortedMap to handle the retrieval of entries based on closest-match searches.
SequencedMap	Defines a special kind of Map whose Map.Entry objects have a well-defined encounter order.
SortedMap	Extends SequencedMap so that the keys are maintained in ascending order.

Map interface

Declaration

```
interface Map<K, V>
```

- $K \rightarrow$ type of key
- $V \rightarrow$ type of value

Method	Description
<code>void clear()</code>	Removes all key/value pairs from the invoking map.
<code>default V compute(K k, BiFunction<? super K, ? super V, ? extends V> func)</code>	Calls <i>func</i> to construct a new value. If <i>func</i> returns non- null , the new key/value pair is added to the map, any preexisting pairing is removed, and the new value is returned. If <i>func</i> returns null , any preexisting pairing is removed, and null is returned.
<code>default V computeIfAbsent(K k, Function<? super K, ? extends V> func)</code>	Returns the value associated with the key <i>k</i> . Otherwise, the value is constructed through a call to <i>func</i> and the pairing is entered into the map and the constructed value is returned. If no value can be constructed, null is returned.
<code>default V computeIfPresent(K k, BiFunction<? super K, ? super V, ? extends V> func)</code>	If <i>k</i> is in the map, a new value is constructed through a call to <i>func</i> and the new value replaces the old value in the map. In this case, the new value is returned. If the value returned by <i>func</i> is null , the existing key and value are removed from the map and null is returned.
<code>boolean containsKey(Object k)</code>	Returns true if the invoking map contains <i>k</i> as a key. Otherwise, returns false .
<code>boolean containsValue(Object v)</code>	Returns true if the map contains <i>v</i> as a value. Otherwise, returns false .

static <K, V> Map<K, V> copyOf(Map<? extends K, ? extends V> <i>from</i>)	Returns a map that contains the same key/value pairs as that specified by <i>from</i> . The returned map is unmodifiable and value-based. Null keys or values are not allowed.
static <K, V> Map.Entry<K, V> entry(K <i>k</i> , V <i>v</i>)	Returns an unmodifiable value-based map entry comprised of the specified key and value. A null key or value is not allowed.
Set<Map.Entry<K, V>> entrySet()	Returns a Set that contains the entries in the map. The set contains objects of type Map.Entry . Thus, this method provides a set-view of the invoking map.
boolean equals(Object <i>obj</i>)	Returns true if <i>obj</i> is a Map and contains the same entries. Otherwise, returns false .
default void forEach(BiConsumer< ? super K, ? super V> <i>action</i>)	Executes <i>action</i> on each element in the invoking map. A ConcurrentModificationException will be thrown if an element is removed during the process.
V get(Object <i>k</i>)	Returns the value associated with the key <i>k</i> . Returns

null if the key is not found.

default V getOrDefault(Object <i>k</i> , V <i>defVal</i>)	Returns the value associated with <i>k</i> if it is in the map. Otherwise, <i>defVal</i> is returned.
int hashCode()	Returns the hash code for the invoking map.
boolean isEmpty()	Returns true if the invoking map is empty. Otherwise, returns false .

Method	Description
Set<K> keySet()	Returns a Set that contains the keys in the invoking map. This method provides a set-view of the keys in the invoking map.
default V merge(K k, V v, BiFunction<? super V, ? super V, ? extends V> func)	If <i>k</i> is not in the map, the pairing <i>k,v</i> is added to the map. In this case, <i>v</i> is returned. Otherwise, <i>func</i> returns a new value based on the old value, the key is updated to use this value, and merge() returns this value. If the value returned by <i>func</i> is null , the existing key and value are removed from the map and null is returned.
static <K, V> Map<K, V> of(<i>parameter-list</i>)	Creates an unmodifiable value-based map containing the entries specified in <i>parameter-list</i> . Null keys or values are not allowed. Many overloaded versions are provided. See the discussion in the text for details.
static <K, V> Map<K, V> ofEntries(Map.Entry<? extends K, ? extends V> ... <i>entries</i>)	Returns an unmodifiable value-based map that contains the key/value mappings described by the entries passed in <i>entries</i> . Null keys or values are not allowed.
V put(K k, V v)	Puts an entry in the invoking map, overwriting any previous value associated with the key. The key and value are <i>k</i> and <i>v</i> , respectively. Returns null if the key did not already exist. Otherwise, the previous value linked to the key is returned.

void putAll(Map<? extends K, ? extends V> m)	Puts all the entries from <i>m</i> into this map.
default V putIfAbsent(K <i>k</i> , V <i>v</i>)	Inserts the key/value pair into the invoking map if this pairing is not already present or if the existing value is null . Returns the old value. The null value is returned when no previous mapping exists or the value is null .
V remove(Object <i>k</i>)	Removes the entry whose key equals <i>k</i> .
default boolean remove(Object <i>k</i> , Object <i>v</i>)	If the key/value pair specified by <i>k</i> and <i>v</i> is in the invoking map, it is removed and true is returned. Otherwise, false is returned.
default boolean replace(K <i>k</i> , V <i>oldV</i> , V <i>newV</i>)	If the key/value pair specified by <i>k</i> and <i>oldV</i> is in the invoking map, the value is replaced by <i>newV</i> and true is returned. Otherwise false is returned.
default V replace(K <i>k</i> , V <i>v</i>)	If the key specified by <i>k</i> is in the invoking map, its

	value is set to <i>v</i> and the previous value is returned. Otherwise, null is returned.
default void replaceAll(BiFunction< ? super K, ? super V, ? extends V> <i>func</i>)	Executes <i>func</i> on each element of the invoking map, replacing the element with the result returned by <i>func</i> . A ConcurrentModificationException will be thrown if an element is removed during the process.

Method	Description
<code>int size()</code>	Returns the number of key/value pairs in the map.
<code>Collection<V> values()</code>	Returns a collection containing the values in the map. This method provides a collection-view of the values in the map.

Table 20-12 The Methods Declared by **Map**

Important methods of Map interface

1. **put(K key, V value)**

Adds key-value pair

```
map.put(1, "Apple");
```

If key exists → value is replaced

2. **get(Object key)**

Returns value for a key

```
System.out.println(map.get(2));
```

Output: Banana

3. **remove(Object key)**

Removes entry

```
Ex: map.remove(3);
```

4. **containsKey(Object key)**

```
System.out.println(map.containsKey(1)); // true
```

5. **containsValue(Object value)**

```
System.out.println(map.containsValue("Apple")); // true
```

6. **size()**

```
System.out.println(map.size());
```

7. **isEmpty()**

```
System.out.println(map.isEmpty());
```

8. **clear()** Removes all elements

```
map.clear();
```

Traversing Map

1. Using **entrySet()**

```
for(Map.Entry<Integer, String> e : map.entrySet()) {  
    System.out.println(e.getKey() + " " + e.getValue());  
}
```

2. Using **keySet()**

```
for(Integer key : map.keySet()) {  
    System.out.println(key + " " + map.get(key));  
}
```

3. Using **values()**

```
for(String value : map.values()) {  
    System.out.println(value);  
}
```

4. Using **forEach()** (Java 8+)

```
map.forEach((k, v) -> System.out.println(k + " " + v));
```

Advanced Methods

1. `getOrElse()`

```
System.out.println(map.getOrElse(5, "Not Found"));
```

2. `putIfAbsent()` `map.putIfAbsent(2, "Orange");` // won't replace existing

3. `compute()` `map.compute(1, (k, v) -> v + " Fruit");`

4. `computeIfAbsent()` `map.computeIfAbsent(4, k -> "Grapes");`

5. `computeIfPresent()` `map.computeIfPresent(2, (k, v) -> v + " Updated");`

6. `merge()`

```
map.merge(2, "New", (oldVal, newVal) -> oldVal + " " + newVal);
```

1. **getOrDefault(key, defaultValue)**

```
System.out.println(map.getOrDefault(5, "Not Found"));
```

- If key exists → return value
- If key NOT exists → return default value

Output:

Not Found

2. **putIfAbsent(key, value)**

```
map.putIfAbsent(2, "Orange");
```

```
map.putIfAbsent(4, "Grapes");
```

- If key exists → DO NOTHING
- If key not exists → INSERT

Before: {1=Apple, 2=Banana, 3=Mango}

After: {1=Apple, 2=Banana, 3=Mango, 4=Grapes}

Key 2 not replaced

Key 4 added

3. **compute(key, function)**

map.compute(1, (k, v) -> v + " Fruit");

- ALWAYS executes function
- Updates value based on function result

Before: 1=Apple

Function: "Apple" → "Apple Fruit"

After: {1=Apple Fruit, 2=Banana, 3=Mango}

If function returns null:

map.compute(1, (k, v) -> null);

Entry is REMOVED

4. **computelfAbsent(key, function)**

```
map.computelfAbsent(4, k -> "Grapes");  
map.computelfAbsent(2, k -> "Orange");
```

- If key NOT present → compute & insert
- If key exists → DO NOTHING

Before:

{1=Apple, 2=Banana, 3=Mango}

After:

{1=Apple, 2=Banana, 3=Mango, 4=Grapes}

Key 2 unchanged

Key 4 added

5. **computeIfPresent(key, function)**

```
map.computeIfPresent(2, (k, v) -> v + " Updated");  
map.computeIfPresent(5, (k, v) -> "Test");
```

- If key EXISTS → update value
- If key NOT exists → DO NOTHING

Before: 2=Banana

After: 2=Banana Updated

Key 5 ignored

6. **merge(key, value, function)**

```
map.merge(2, "New", (oldVal, newVal) -> oldVal + " " + newVal);  
map.merge(5, "Grapes", (oldVal, newVal) -> oldVal + newVal);
```

Case 1: Key EXISTS

oldVal = Banana

newVal = New

Result: Banana New

Case 2: Key NOT EXISTS Simply INSERT

5=Grapes

Final Map: {1=Apple, 2=Banana New, 3=Mango, 5=Grapes}

```
import java.util.*;

public class FullMapExample {

    public static void main(String[] args) {

        Map<Integer, String> map = new HashMap<>();

        map.put(1, "Apple");

        map.put(2, "Banana");

        map.put(3, "Mango");

        map.putIfAbsent(2, "Orange");

        map.compute(1, (k, v) -> v + " Fruit");

        map.computeIfAbsent(4, k -> "Grapes");
```

```
map.computeIfPresent(2, (k, v) -> v + " Updated");  
    map.merge(3, " Juice", (oldVal, newVal) -> oldVal +  
newVal);  
    map.forEach((k, v) -> System.out.println(k + " : " + v));  
}  
}
```

```
import java.util.*;

public class FullMapExample {

    public static void main(String[] args) {

        Map<Integer, String> map = new HashMap<>();

        map.put(1, "Apple");
        map.put(2, "Banana");
        map.put(3, "Mango");
        map.putIfAbsent(2, "Orange");
        map.compute(1, (k, v) -> v + " Fruit");
        map.computeIfAbsent(4, k -> "Grapes");
        map.computeIfPresent(2, (k, v) -> v + " Updated");
        map.merge(3, " Juice", (oldVal, newVal) -> oldVal + newVal);
```

```
// Traversal using Map.Entry  
for (Map.Entry<Integer, String> entry : map.entrySet()) {  
    System.out.println(entry.getKey() + " : " + entry.getValue());  
}  
}  
}
```

Output:

1 : Apple Fruit
2 : Banana Updated
3 : Mango Juice
4 : Grapes

Map.Entry Interface

The **Map.Entry Interface** represents **one key–value pair inside a Map**.

Think of it like: (**key** , **value**)

Where do we use it?

From a Map:

map.entrySet() Returns a **Set of Map.Entry objects**

Map.Entry represents a **single key-value pair** and is used for **efficient traversal, updating, and sorting of maps**.

Important Methods of Map.Entry

1. **getKey()**

`entry.getKey();`

Returns the key

2. **getValue()**

`entry.getValue();`

Returns the value

3. **setValue(V value)**

`entry.setValue("Orange");`

Updates value of that entry

Example (Updating values)

```
for (Map.Entry<Integer, String> entry : map.entrySet()) {  
    if (entry.getKey() == 2) {  
        entry.setValue("Orange");  
    }  
}
```

Output:

1 : Apple
2 : Orange
3 : Mango

Method	Description
boolean equals(Object <i>obj</i>)	Returns true if <i>obj</i> is a Map.Entry whose key and value are equal to that of the invoking object.
K getKey()	Returns the key for this map entry.
V getValue()	Returns the value for this map entry.
int hashCode()	Returns the hash code for this map entry.
V setValue(V <i>v</i>)	Sets the value for this map entry to <i>v</i> . A ClassCastException is thrown if <i>v</i> is not the correct type for the map. An IllegalArgumentException is thrown if there is a problem with <i>v</i> . A NullPointerException is thrown if <i>v</i> is null and the map does not permit null keys. An UnsupportedOperationException is thrown if the map cannot be changed.

Table 20-16 The Non-Static Methods Declared by **Map.Entry**

```
import java.util.*;

public class SortByKey {

    public static void main(String[] args) {

        Map<Integer, String> map = new HashMap<>();

        map.put(3, "Mango");

        map.put(1, "Apple");

        map.put(2, "Banana");

        List<Map.Entry<Integer, String>> list = new ArrayList<>(map.entrySet());

        list.sort(Map.Entry.comparingByKey());

        for (Map.Entry<Integer, String> e : list) {

            System.out.println(e.getKey() + " : " + e.getValue());  }  } }
```

Static Methods of **Map.Entry**

1. **comparingByKey()**

Used for **sorting based on keys**

Output

```
1 : Apple
2 : Banana
3 : Mango
```

2. **comparingByValue()**

Used for **sorting based on values**

Example

```
list.sort(Map.Entry.comparingByValue());
```

Output

1 : Apple

2 : Banana

3 : Mango

(Alphabetical order of values)

3. **copyOf()** (JDK 17)

Creates **immutable (read-only) copy of entry**

Example

```
Map.Entry<Integer, String> e1 = Map.entry(1, "Apple");
```

```
Map.Entry<Integer, String> e2 = Map.Entry.copyOf(e1);
```

```
System.out.println(e2.getKey() + " " + e2.getValue());
```

Important

```
e2.setValue("Orange"); // UnsupportedOperationException
```

Because it is unmodifiable

The Map Classes

Class	Description
AbstractMap	Implements most of the Map interface
EnumMap	Extends AbstractMap for use with enum keys
HashMap	Extends AbstractMap using a hash table to implement the Map methods
TreeMap	Extends AbstractMap using a tree implementation to be a SortedMap
WeakHashMap	Extends AbstractMap to use a hash table with weak keys that may be garbage-collected
LinkedHashMap	Extends HashMap so that this is also a SequencedMap implementation
IdentityHashMap	Extends AbstractMap and uses reference equality when comparing documents

HashMap

The **HashMap** class is part of Java Collections Framework.

It stores data in **key** → **value pairs using a hash table**

Key Features

- Keys are **unique**
- Values can be **duplicate**
- **Fast operations** → put() and get() are almost **O(1)**
- **No order guarantee** (random order)
- Allows:
 - One null key
 - Multiple null values

HashMap uses **hashing**

Key → **hashCode()** → **index** → **store value**

This is why it is very fast

```
Map<String, String> map = new HashMap<>();
```

```
map.put(null, "A");
```

```
map.put(null, "B");      // replaces previous value
```

```
map.put("X", null);
```

```
map.put("Y", null);      // multiple null values allowed
```

Constructors

HashMap() Default capacity = **16**, load factor = **0.75**

HashMap(int capacity) Custom initial size

HashMap(int capacity, float loadFactor) Custom size + fill ratio

HashMap(Map m) Copy another map

Capacity = number of buckets (slots) in HashMap

Think of HashMap like a set of boxes:

[] [] [] [] ... (16 boxes by default)

Default capacity = **16**

Load Factor = how full the HashMap is allowed to get before resizing

Default = **0.75 (75%)**

What does 0.75 mean?

Resize happens when:

capacity × load factor

Example:

capacity = 16

load factor = 0.75

threshold = $16 \times 0.75 = 12$

When you insert the **13th element**, HashMap will:

Resize (increase capacity)- means When limit is reached: Capacity
doubles $16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \dots$

```
HashMap<Integer, String> map = new HashMap<>(32);
```

Meaning:

- Initial capacity = **32**
- Load factor = **0.75 (default)**

Resize after:

$32 \times 0.75 = 24$ elements

Important Note

HashMap **does NOT** maintain order

```
map.put(1, "A");  
map.put(2, "B");
```

Output may be:

```
{2=B, 1=A}
```

```
import java.util.*;

public class HashMapDemo {

    public static void main(String[] args) {

        HashMap<String, Double> map = new HashMap<>();
        map.put("John", 1000.0);
        map.put("Jane", 2000.0);
        map.put("Sam", 1500.0);
        for (Map.Entry<String, Double> e : map.entrySet()) {
            System.out.println(e.getKey() + " : " + e.getValue());
        }
    }
}
```

Output (order not fixed)

John : 1000.0

Jane : 2000.0

Sam : 1500.0

Real Example (Bank Account)

```
import java.util.*;
public class BankExample {
    public static void main(String[] args) {
        HashMap<String, Double> accounts = new HashMap<>();
        accounts.put("John", 1000.0);
        accounts.put("Jane", 2000.0);
        // Deposit into John's account
        double balance = accounts.get("John");
        accounts.put("John", balance + 500);
        for (Map.Entry<String, Double> e : accounts.entrySet()) {
            System.out.println(e.getKey() + " : " + e.getValue());
        }
    }
}
```

Output

John : 1500.0
Jane : 2000.0

Write a Java program to store student names and their marks using a HashMap.

Perform the following operations:

- Add at least 5 students
- Display all entries
- Search for a student by name
- Update marks of a student

Create a program that:

- Stores email IDs using HashSet
- Prevents duplicate registrations

Create a HashMap to store employee ID and salary.

Perform:

- Insert records
- Find highest salary
- Remove an employee